

AI 编码协作 12 条执行规则

根据用户提供截图整理，用于约束 AI 编码助手在需求澄清、代码修改、验证交付中的执行方式。

核心目标

减少“猜需求、过度设计、改 A 坏 B、验证不足”的错误。每次编码任务都要先明确假设，再做最小必要修改，并在每个阶段留下可验证的检查点。

一、适用场景

- 使用 Claude Code、Codex、Cursor、Copilot Chat 等 AI 编码助手进行需求实现、缺陷修复、重构或测试补齐。
- 多人协作项目中，需要避免 AI 越界修改、重构他人代码、或在不确定条件下强行推进。
- 长会话、大任务、多模块改动场景中，需要控制上下文、Token、验证路径和失败记录。

二、12 条规则总览

序号	规则	行动口径	避免事项
1	先想再写	动手前先列出目标、输入、约束、风险和关键假设；不确定就问清楚。	不要把猜测当事实，不要在需求含糊时直接改代码。
2	简单优先	优先用最少代码解决当前问题，复用项目已有模式、工具和接口。	不要为了显得完整而做超前抽象、平台化或大而全设计。
3	外科手术式修改	只改与目标直接相关的位置，保留他人代码和无关行为。	不要顺手重构、格式化大片文件，或把局部修复扩成全局改造。
4	目标驱动执行	先定义成功标准，再执行修改；验证通过后才算完成。	不要只因为代码写完就宣布完成，也不要跳过验证。
5	模型只做判断	确定性逻辑交给代码、规则、路由、测试和重试机制；LLM 负责语义判断和不确定推理。	不要让 LLM 做路径选择、重试循环、状态机、校验规则等可

序号	规则	行动口径	避免事项
			编程逻辑。
6	硬 Token 预算	单任务控制在约 4K token，会话控制在约 30K token；大任务拆成小任务。	不要把调试日志、全量文件和无关上下文无限塞进会话。
7	冲突选一边	同一处问题只采用一种方案：要么修补当前设计，要么切换新设计。	不要混合两种架构、两套状态、两种数据来源或两种 UI 模式。
8	先读再写	先理解导出方、调用方、共享工具和测试，再动手修改。	不要只看报错行就改；不要没读上下游就改公共接口。
9	测试验证意图	测试要说明为什么这样做，覆盖行为、边界和回归风险。	不要只测“做了什么”，也不要写与需求无关的脆弱快照测试。
10	每步检查点	每个阶段总结：做了什么、验证了什么、还剩什么。	不要长时间沉默推进，也不要把所有风险堆到最后才暴露。
11	遵循代码库惯例	项目用什么框架、命名、错误处理、数据流，就沿用什么。	不要把蛇形项目改成驼峰风格，或引入项目没有的范式。
12	失败要大声	失败、超时、跳过、无法验证都要明确记录，并说明影响范围。	不要在不确定时写“完成”；不要掩盖失败或吞掉异常。

三、执行流程模板

1. **任务理解**：复述目标、边界、成功标准；列出不确定点。
2. **代码阅读**：定位入口、调用链、共享工具、现有测试和相关约定。
3. **修改计划**：说明将改哪些文件、为什么只改这些位置、风险在哪里。
4. **小步实现**：按最小可验证单元修改，避免把无关重构混进同一批。
5. **验证反馈**：运行最相关测试；必要时补测试；记录失败和未覆盖风险。
6. **交付总结**：说清楚改了什么、验证了什么、还可以继续优化什么。

四、任务前检查清单

检查	检查点	通过标准
☐	假设已写清	任务目标、输入条件、限制范围和不确定点已经明示。
☐	范围已收窄	只改与目标相关的文件，不混入顺手重构。
☐	方案只选一种	没有同时保留两套互相竞争的实现路径。
☐	上下游已阅读	调用方、导出方、共享工具和现有测试已经看过。
☐	验证已定义	知道要跑哪些测试或手动检查，无法验证时会说明原因。
☐	失败会记录	超时、跳过、异常、缓存命中/未命中都会在结果中交代。

五、可直接放进提示词的版本

提示词片段

在开始编码前，请先说明你的假设、成功标准和将阅读的文件；不确定就先问。实现时优先选择最简单方案，只做与目标相关的外科手术式修改，遵循当前代码库惯例。确定性逻辑用代码、测试、路由和重试机制完成，不要交给 LLM 猜。每完成一个阶段都说明做了什么、验证了什么、还剩什么。如果失败、超时或无法验证，请明确说出来，不要把不确定结果写成完成。

六、使用建议

这 12 条规则最适合放在项目级 AI 协作规范、仓库的开发约定、或单次任务的系统提示中。若任务很小，可以只使用“先想再写、简单优先、外科手术式修改、目标驱动执行”四条；若任务跨模块或长会话，则必须启用完整 12 条。